GROUP WORK PROJECT # _2__ MScFE 660: RISK MANAGEMENT
GROUP NUMBER: ___16497_____

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Ge Zhang	USA	gzhang8@gmail.com	
Alberto Hernandez-Espinosa		albertohernandezespinosa@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an "X" above).	
Team member 1	**Ge Zhang**
Team member 2	**Alberto Hernandez-Espinosa**
Team member 3	

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed. **Note:** You may be required to provide proof of your outreach to non-contributing members upon request.

Step 1:

A Hidden Markov Model is a 5-tuple (Q,∑,Π,A,B), where Q = {q1,⋯ ,qN} is a finite set of N states, ∑ = {s1,⋯ ,sN} is the set of M possible symbols (emissions) in the language, Π ={πi}is the initial probability vector, A = {aij} is the state transition probability matrix, and B ={bi(vk)} is the emission probability matrix. The HMM can be denoted by λ = (Π,A,B).

Specifically or detecting regimes in time-series data such as oil prices, a Hidden Markov Model with two emission sequences, i.e., the difference between consecutive months being the symbols ∑ (1- increase / 0- decrease), and three hidden states, Q, (bull, bear, stagnant market regimes), is estimated using forward/backward algorithm, Viterbi algorithm, and the Baum-Welch algorithm.

Forward algorithm:

Evaluating the probability of an observed sequence of emissions $O = o_1 o_2 o_3 \dots o_T$ ($o_i \in \Sigma$), given a particular HMM λ = (Π,A,B), the forward variable is calculated as:

$$\alpha_t(i) = p(o_1, o_2, \dots o_t, q_t = s_i|\lambda)$$

i.e. the probability that the partially observed sequence up to time t have been generated and the system is in state s_i at time t.

Base case: $\alpha_1(i) = \pi_i b_i(o_1)$ (Having initial state as q_0 and generating o_1).

General case: $\alpha_{t+1}(i) = b_i(o_{t+1}) \sum_{j=1}^{N} \alpha_t(j).a_{ji}$, where 1 ≤ t < T (Where $\alpha_1(i), \alpha_2(i), \cdots$, $\alpha_T(i)$ corresponds to the T observed symbols. We observe that $p(O|\lambda) = \sum_{i=0}^{N} \alpha_t(j)$.

Backward algorithm:

For training the HMMs, we require a second set of probabilities- the β values.

Just like for the α values, we define the backward variable $\beta_t(i)$ as

$$\beta_t(i) = p(o_{t+1}, o_{t+2}, \dots o_T, q_t = s_i|\lambda)$$

i.e. the probability of the observed sequence from (t + 1) to T given the state s_i in time t.

Base Case: $\beta_T(i) = 1$ (Since we have no symbol to generate, we assume each state could be a possible halting state).

General Case: $\beta_t = \sum_{i=0}^{N} \beta_{t+1}(j).\, a_{ij}.\, b_j(o_{t+1})$, where 1 ≤ t < T (o_{t+1} can be generated from any state s_i.

We observe that $p(O|\lambda) = \sum_{i=0}^{N} \alpha_1(i)\beta_1(i)$ and as an inductive corollary, can be extended to $p(O|\lambda) = \sum_{i=0}^{N} \alpha_t(i)\beta_t(i)$, where 1 ≤ t ≤ T .

The Baum-Welch Algorithm :

The Baum-Welch algorithm is an Expectation-Maximisation (EM) algorithm for HMMs. It starts with a random guess of the parameter values, and iteratively computes the expected probability of all possible transition paths and then re-estimates all parameters based on the expected counts of every corresponding event. This processe is repeated until the likelihood converges.

The process of updating formulae can be expressed in terms of the α's and β's together with the current parameter values.

Let $\gamma_t(i) = p(q_t = s_i | O, \lambda)$ be the probability of being in state s_i at time t, given O (the observation sequence), and $\xi_t(i,j) = p(q_t = s_i, q_{t+1} = s_j | O, \lambda)$ be the transition probability from state s_i to state s_j given the O observation sequence.

$\gamma_t(i) = \sum_{j=1}^{N} \xi_t(i, j)$, and moreover, for t = 1,⋯ ,T, $\gamma_t(i) = \frac{\alpha_t(i)\beta_t(i)}{\sum_{j=0}^{N} \alpha_t(i)\beta_t(i)}$

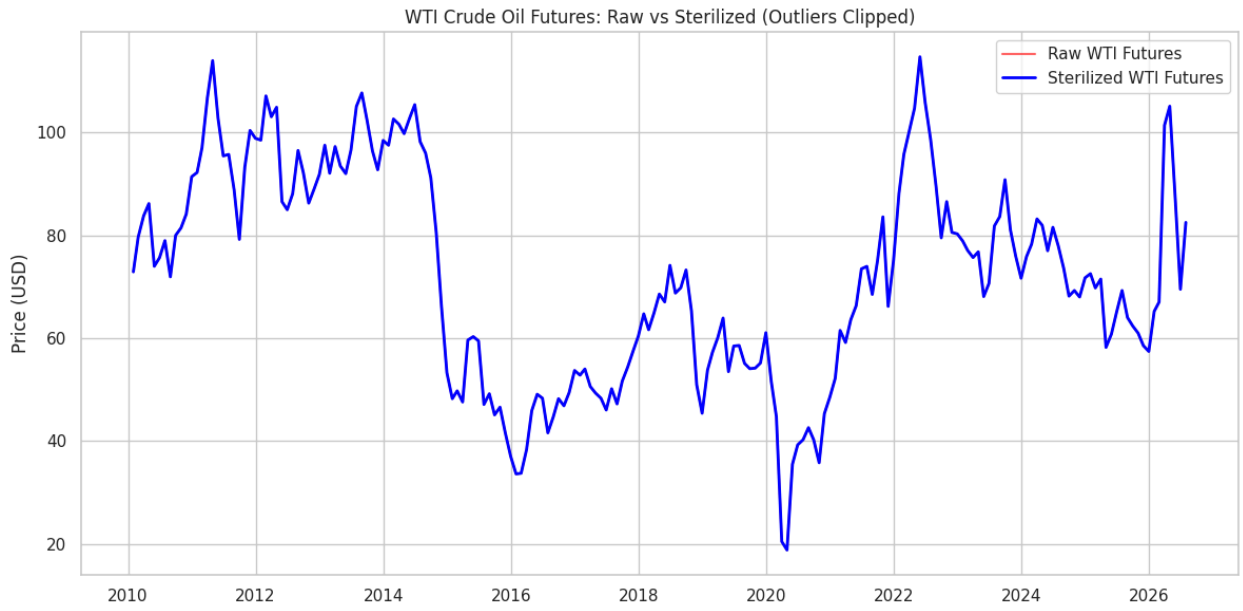
The formulae for updating all parameters are:

• $\pi'_i = \gamma_1(i)$

- $a_{ij}' = \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{j=1}^{N} \sum_{t=1}^{T-1} \xi_t(i,j)}$

- $b_i'(v_k) = \frac{\sum_{t=1, o_t=v_k}^{T-1} \gamma_t(i,)}{\sum_{t=1}^{T} \gamma_t(i)}$

Step 2:



Take WTI Crude Oil price as an example, the time series from year 2021 to 2023 is in bull regime, and the time period from year 2015 to 2016 is in bear regime.

Step 3:

A Hidden Markov Model (HMM) is a probabilistic model consisting of a Markov chain of unobserved (hidden) states and a set of observable emissions generated by those states. The hidden states follow the Markov property, and each observation depends only on the current hidden state.

An HMM is typically defined by:

$$\lambda = (\pi, A, B),$$

In which,

π = initial state probability distribution

A = state transition probability matrix, $a_{ij} = p(q_{t+1} = j | q_t = i)$

B = emission probability distribution, $b_j(o) = p(o_t = o | q_t = j)$

Example: Market Regimes

Hidden states: Bull market, Bear market, Stagnant market

Observations: Daily returns, Volatilities

The observed emissions are returns and volatility, and the hidden states are the true market regimes. The HMM infers the most likely regime sequence from the observed data.

Step 4:

The global macro strategy relating to oil trading uses Macroeconomic indicators as buy and sell signals on a monthly scale. These macroeconomic variables are usually economic time-series data provided by the Energy Information Administration (EIA) for data for the energy markets and the Federal Reserve Economic Data (FRED) for macroeconomic factors. We would be using these EIA forecasts as inputs for inference on our learnt model to obtain the forecast of the price of oil as the output.

The EIA assesses the various factors that influence crude oil prices; physical market factors as well as those relating to trading and financial markets. The EIA lists non-OPEC and OPEC crude oil production, non-OECD and OECD consumption of crude oil, OECD inventories, proven reserves, geopolitical events, and the trading of oil in financial markets as the main factors affecting the price of oil.

We also used data from the FRED, which includes a number of macroeconomic variables that played a pivotal role in the oil markets and the price of crude oil. For example, the monetary policy dictated by the Federal Reserve Bank, the macroeconomic atmosphere of the economy, industrial growth, industrial production, capacity, and capacity utilization.

Data preprocessing is the first step of removing data inconsistencies such as missing values, noise, outliers, and unformatted data. There are a number of available Python libraries for retrieving data from the EIA and FRED. The data retrieved from the EIA and FRED are often unable to be used directly due to their unstructured format, we would be using pandas for data preprocessing and noise-reduction, and transform data into formats which could be used with Python libraries pgmpy and hmms. The final product of data preprocessing is the training dataset, which would be used to train the model, as well as the testing dataset for assessing the performance of our model.

Before we decide to split our preprocessed data into training, testing, and validation datasets, there is a vital step that needs to be carried out. The pandas DataFrame that pgmpy takes in as input has values of a discretised nature i.e. having a fixed number of states. The data obtained from the EIA and FRED is raw continuous time-series data which has almost an infinite number of states. In order for pgmpy to use this data, it is essential that we find a mechanism to find hidden states in time-series data for discretisation of this data. We would apply the Hidden Markov Model to discretise the EIA and FRED data into three regimes.

The general idea of systematic oil trading is to use macroeconomic and physical market data to identify opportune times to be long in the oil markets, and periods when it is beneficial to be short. We would prefer to be long before periods where oil markets are in a bull regime and be short before periods of a bear regime.

Regime detection is the process by which we identify periods of similar volatility in a time series model. These periods could either be bull/bear regimes, periods of low or high volatility, or perhaps some other characteristic that cannot be qualitatively described. These different regime periods are latent in nature and can only be observed as emissions (returns), the Hidden Markov Models were constructed to detect regimes in oil prices.

The HMM would be a 5-tuple (Q,Σ,Π,A,B), and the regime detection process comprises four steps:

1. Transforming the time-series into an emission sequence by first-order integration of the concerned time-series and replacing the returns with the parity of the returns.

2. Learning the parameters of the assumed HMM generates the time series by using the Baum-Welch algorithm, hence allowing us to obtain the Π (initial state probabilities), A (state transition probability matrix), and B (emission matrix).

3. Finding the most likely sequence of hidden states as the underlying regimes that possibly generated the sequence of observed emissions using the Viterbi algorithm.

4. Identifying the latent meaning behind each hidden state by indexing them according to their arithmetic mean.

The Python implementation for HMMs provides ready-to-use implementations of the Forward-Backward, Viterbi and Baum-Welch algorithms.

Belief networks are highly applicable to capture causal relationships between the standard data-driven economic variables and quantified expert judgements about the geo-politics of the oil market. A popular

score based structure learning algorithm is Hill Climbing, a greedy, iterative search algorithm used to maximize the network score.

Python library pgmpy provides a straightforward implementation to the Hill Climbing algorithm. The pandas.DataFrame that pgmpy takes in as input must have discrete values of a fixed number of states. The data obtained from the EIA and FRED is raw continuous time-series data. In order for pgmpy to use this data, we have to call HMM to find hidden regimes in the EIA and FRED time-series data for discretisation of this data.

The first step of constructing our model is to retrieve the EIA and FRED data from the open-data facilities. Unfortunately, both these open data facilities do not provide Python packages to neatly retrieve data in Python, so we will have to resort to using third-party APIs. For the EIA, we are using EIA-python, and for the FRED we are using fredapi. Below code retrieves data from EIA API.

```
# Importing the library

import eia

# the API key we recieved from EIA

eia_key = "XXXXXXXXXX"

# Initiates a session with the EIA datacenter to receive

datasets eia_api = eia.API(eia_key)

# Convert to pandas dataframe

eia_data = pd.DataFrame(eia_api.data_by_series(series='TOTAL.COEXPUS.M'))
```

The data cleaning process includes converting the index of EIA dataframe to pandas standard format datetime, and replacing the holes in the data i.e. rows marked by a ‘-’ (a single dash) by np.nan so that we can use pandas to fill in the holes.

Similar to EIA-python, the fredapi requires a registration with the FRED API so that we can download the Spot Crude Oil Price.

```
from fredapi import Fred

# FRED API key

fred_key = "XXXXXXXXX"

# Initiates a session with the FRED datacenter

datasets fred = Fred(api_key=fred_key)
```

```
# Retrieve data from FRED API

fred_data = pd.DataFrame(fred.get_series('WTISPLC'), columns=['WTISPLC'])
```

To apply Belief networks, we have to reduce our data from prices to a set of states, such as bull, bear, and stagnant markets. In order to detect these (hidden) states, we have to use Hidden Markov Models for Regime Detection in time series price data.

Step 6:

We would be using a Python library called hmms for implementing the Hidden Markov Models. For detecting regimes in time-series data, we would be using Hidden Markov Models, with the difference between consecutive months being the symbols ∑ (1- increase / 0- decrease), the hidden states, Q being the bull, bear, stagnant market regimes. For example, for 'WTISPL' (Spot Crude Oil Price: West Texas Intermediate), below code is applied to identify regimes in the time series:

```
import hmms

price = train_data['WTISPLC']

price_diff = price.diff()[1:]

# Replacing the change with 1 if positive, else 0

e_seq = np.array(price_diff.apply(lambda x: 1 if x > 0 else 0).values)
```

Given that we have obtained the output (observed emission sequence), we can now use the Baum-Welch algorithm to learn the parameters of the HMM generating this data. We apply the training data to train the HMM and validate predictions on the validation dataset and will tune the model parameters to fit the validation. Finally we would be using the testing dataframe to test the final performance of the tuned model after validation.

We will start with random parameters to create a discrete time HMM of three hidden state (bull,bear, stagnant) and two output variables (increase or decrease). The parameters will be eventually trained to maximize the fit to training data.

```
dhmm_r = hmms.DtHMM.random(3 , 2)
```

Given that the hmms.DtHMM takes a list of arrays no greater than length 32, we will have to split our array in arrays each of length 32 or less.

```
e_seq = np.array_split(e_seq, 32)
```

We now use the Baum-Welch algorithm to learn the parameters of the HMM to detect regimes.

```
 # 100 iterations

dhmm_r.baum_welch(e_seq, 100)

hmms.print_parameters(dhmm_r )
```

Given parameters λ for HMM and the emitted observation sequence e_seq,we can use the Viterbi Algorithm to identify the most likely state-transition path (i.e.market regimes) in the financial time-series.

```
(log_prob, s_seq)= dhmm_r.viterbi(np.concatenate(e_seq).ravel())
```

We will be plotting this graph in a multicolored time-series plot to observe how well the regimes have been identified.

```
# Add price

price_plot = pd.DataFrame(price[1:], index=price[1:].index)

# Add a column representing the regime

price_plot['Regime'] = s_seq

# Add a column representing the increase or decrease in price

price_plot['diff'] = price_diff
```

Given that the bull regimes should have a high positive change in price, bear regimes should have a high negative change, and stagnant regimes are closer to zero, we can use these properties to tell which state represents which regime.

```
# Get means of all assigned states

means = price_plot.groupby(['Regime'])['diff'].mean()

lst_1 = means.index.tolist()

lst_2 = means.sort_values().index.tolist()

map_regimes = dict(zip(lst_2, lst_1))

price_plot['Regime'] = price_plot['Regime'].map(map_regimes)
```

Plotting the data as a multi-colored time series. We can observe from the graph that there are periods of bear runs, bull runs, and stagnant periods. In order to verify if the HMM has correctly identified regimes in all datasets, we should plot the regime-switching models of all datasets and observe if the bull, bear,

and stagnant states have been correctly identified. The graphical representation allows us to understand how the regimes have been learnt.

```
for series_id in datasets:
        df = pd.DataFrame(index=train_data[1:].index);
        df[series_id] = train_data[series_id][1:];
        df['Diff'] = train_data[series_id].diff()[1:];
        df['Regime'] = states[series_id];
        # Get means of all assigned states
        means = df.groupby(['Regime'])['Diff'].mean();
        lst_1 = means.index.tolist();
        lst_2 = means.sort_values().index.tolist();
        map_regimes = dict(zip(lst_2, lst_1));
        df['Regime'] = df['Regime'].map(map_regimes);
        cmap = ListedColormap(['r','b','g'],'indexed');
        norm = BoundaryNorm(range(3 + 1), cmap.N);
        inxval = mdates.date2num(df[series_id].index.to_pydatetime());
        points = np.array([inxval, df[series_id]]).T.reshape(-1, 1, 2);
        segments = np.concatenate([points[:-1], points[1:]], axis=1);
        lc = LineCollection(segments, cmap=cmap, norm=norm);
        lc.set_array(df['Regime']);
        plt.gca().add_collection(lc);
        plt.xlim(df[series_id].index.min(), df[series_id].index.max());
        plt.ylim(df[series_id].min(), df[series_id].max());
        r_patch = mpatches.Patch(color='red', label='Bear');
        g_patch = mpatches.Patch(color='green', label='Bull');
        b_patch = mpatches.Patch(color='blue', label='Stagnant');
        plt.legend(handles=[r_patch, g_patch, b_patch]);
```

```
name = "./plots/" + series_id.replace(".", "_") + ".png";

plt.savefig(name);

plt.close();
```

Step 7:

The general idea of systematic oil trading is to use macroeconomic and physical market data to identify the structure of the markets, thus assisting us in identifying opportune times to be long in the oil markets, and periods when it is beneficial to be short. As a matter of fact, it is obvious that we would prefer to be long before periods where oil markets are in a bull regime and be short before periods of a bear regime.

Regime detection is the process by which we identify periods of similar volatility in a time series model. These periods could either be bull/bear regimes, periods of low or high volatility, or perhaps some other characteristic that cannot be qualitatively described. Given that these periods are latent in nature and can only be observed as emissions (returns), the problem ultimately decomposes to the problem Hidden Markov Models were constructed to detect.

Step 8:

Given that we have discretised our training dataset, we can now use pgmpy to construct a belief network of all the variables. We would be using the Hill Climbing approach to learn the belief network.

```
from pgmpy.models import BayesianModel

from pgmpy.estimators import HillClimbSearch

from pgmpy.estimators import BayesianEstimator

from pgmpy.estimators import BicScore, K2Score, BdeuScore

 # Retrieve training set

train_data = pd.read_csv("./data/train_data.csv", index_col=0);


# Initialise Hill Climbing Estimator

hc = HillClimbSearch(train_data, scoring_method=K2Score(train_data));

expert = BayesianModel();

expert.add_nodes_from(datasets);
```

```
        expert.add_edges_from([ ('STEO.PAPR_NONOPEC.M', 'WTISPLC'), ('STEO.PAPR_OPEC.M',
'WTISPLC'), ('STEO.PATC_OECD.M', 'WTISPLC'), ('STEO.PATC_NON_OECD.M', 'WTISPLC'),
('STEO.RGDPQ_OECD.M', 'STEO.PATC_OECD.M'), ('STEO.RGDPQ_NONOECD.M',
'STEO.PATC_NON_OECD.M'), ]);

        model = hc.estimate(expert);

        # Performs local hill climb search

        model.fit(train_data, state_names=dict(map(lambda e: (e, [0, 1, 2]), datasets)),

                estimator=BayesianEstimator, prior_type="K2")
```

Step 9:

Implementation for HMMS, as HMMS is not available on google colab, we replace it with hmmlearn CategoricalHMM.

```
from hmmlearn.hmm import CategoricalHMM

model = CategoricalHMM(

   n_components=3,

   n_iter=100

)

data_diff = df_clean['WTI_Spot'].diff()[1:]

emit_seq = data_diff.apply(lambda x: 1 if x > 0 else 0)

emit_seq_2 = emit_seq.to_frame(name='emit_seq')


model.fit(emit_seq_2)


model.emissionprob_

array([[0.3140701 , 0.6859299 ],

      [0.44001601, 0.55998399],

      [0.77688166, 0.22311834]])
```

model.transmat_

array([[7.46343229e-01, 1.28849772e-03, 2.52368273e-01],

[1.88221353e-04, 4.13353799e-01, 5.86457980e-01],

[4.48028417e-01, 1.74263483e-02, 5.34545235e-01]])

model.startprob_

array([1.00000000e+00, 6.78424632e-15, 1.47468737e-11])

states = model.predict(emit_seq_2)

Implementation for Hill Climb Search:

from pgmpy.estimators import HillClimbSearch

from pgmpy.models import DiscreteBayesianNetwork

data: pandas DataFrame

hc = HillClimbSearch(df_clean)

best_model = hc.estimate(scoring_method="bic-g")

best_model.edges()

OutEdgeView([('Brent_BZ', 'USD_Idx'), ('Brent_BZ', 'Ind_Prod'), ('WTI_CL', 'Brent_BZ'), ('WTI_CL', 'Ind_Prod'), ('USD_Idx', 'Ind_Prod'), ('CPI', 'Fed_Funds'), ('CPI', 'USD_Idx'), ('CPI', 'WTI_CL'), ('CPI', 'Brent_BZ'), ('Fed_Funds', 'USD_Idx'), ('WTI_Spot', 'WTI_CL')])

```
import networkx as nx
import pylab as plt
G=nx.Graph();
G.add_edges_from(best_model.edges());
pos = nx.spring_layout(G);
nx.draw_networkx_nodes(G, pos, node_size = 10);
nx.draw_networkx_edges(G, pos, arrows=True);
plt.figure(5,figsize=(20,10))
```

Bayesian Network